Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ” Python in web”

A efectuat : st. gr.SI – 263 Gonța Marius

A verificat : asistent universitar Patricia Reițman

Chișinău – 2026

Introducere

Prezenta lucrare de laborator are drept scop consolidarea cunoștințelor practice în programarea cu limbajul Python, punând accent pe interacțiunea automatizată cu mediul web, extragerea datelor și analiza de rețea. Pe parcursul a cinci capitole structurate progresiv, lucrarea urmărește tranziția de la concepte fundamentale – precum manipularea corectă a pachetelor standard și externe (ex. requests, urllib) sau utilizarea metodelor specifice obiectelor HTTP – la arhitecturarea unui cod modular și reutilizabil.

Obiectivul principal constă în asamblarea tuturor conceptelor studiate într-un modul propriu (webtools.py), integrat ulterior într-un utilitar executabil din linia de comandă. Prin implementarea cererilor HTTP controlate, gestionarea erorilor de conexiune și analiza elementelor criptografice sau de infrastructură (certificate SSL, rezoluție DNS, antete de securitate), lucrarea simulează un scenariu real de analiză a unui domeniu web. Această abordare practică este esențială pentru înțelegerea profundă a protocoalelor de comunicare online și reprezintă o bază solidă pentru dezvoltarea instrumentelor software dedicate securității și ingineriei informaționale.

Capitolul 1: Importuri

Exercițiul 1. Primul import: Acest exercițiu prezintă importul modulelor requests și urllib.request. De asemenea, include un comentariu explicativ referitor la instalarea manuală a bibliotecilor externe folosind utilitarul pip.

```
# Exercitiul 1
import requests
print(requests.__version__)
import urllib.request #urllib este biblioteca interna Python, requests este biblioteca externa care trebuie instalata
```

Figura 1.1 – „ImportulRequestsURLLib”

Exercițiul 2. Două stiluri de import: Realizarea cererilor HTTP și afișarea codului de stare, evidențiind diferența vizuală și sintactică dintre utilizarea clasică import requests și apelul direct prin from requests import get.

```
# Exercitiul 2
response=requests.get(BASE_URL, timeout=TIMEOUT)  #importeaza toata biblioteca requests, mai clara originea functiei
print(response.status_code)

from requests import get
response=get(BASE_URL, timeout=TIMEOUT)           #importeaza doar functia get, sintaxa mai scurta
print(response.status_code)
```

Figura 1.2 – „StiluriImportRequests”

Exercițiul 3. Alias-uri: Importarea bibliotecii requests sub forma unui alias scurt (rq), cu explicații privind situațiile în care acest stil face codul mai greu sau mai ușor de citit.

Figura 1.3 – „UtilizareAliasRQ”

```
# Exercitiul 3
import requests as rq
response=rq.get(BASE_URL, timeout=TIMEOUT)        #aliasul usureaza citirea in cazul in care este folosita prescurtarea intuitiva
print(response.status_code)                       #a cuvantului, de exemplu rq este evident prescurtarea requests, in caz contrar
                                                  #aliasul va fi inteles doar de cel care il predefineste
```

Exercițiul 4. Doar biblioteca standard: Accesarea site-ului țintă folosind exclusiv pachetul nativ urllib.request.urlopen(), urmată de decodarea setului de octeți utilizând formatul "utf-8".

```
# Exercitiul 4
import urllib.request
try:
    response=urllib.request.urlopen(BASE_URL, timeout=TIMEOUT)
    print(response.status)
    caractere=response.read().decode("utf-8")
    print(caractere[:200])
except Exception as e:
    print(f"Eroare la accesarea URL-ului: {e}")
```

Figura 1.4 – „CerereURLLibNativa”

Exercițiul 5. Priviți în interiorul unui modul: Utilizarea funcției native dir() pentru a inspecta și vizualiza toate atributele și metodele din interiorul modulului requests, alături de identificarea tipurilor de componente (clase, module, funcții).

```
# Exercitiul 5
print(dir(requests)) #ConnectTimeout este o clasa, status_codes este un modul si  get este o functie
```

Figura 1.5 – „InspectareModulDir”

Exercițiul 6. Citiți documentația: Consultarea modului de utilizare din documentația oficială cu ajutorul funcției help() și setarea cu succes a parametrului de limită de timp (timeout=TIMEOUT) pentru evitarea blocajelor.

```
# Exercitiul 6
help(requests.get)
response=requests.get(BASE_URL,timeout=TIMEOUT)
print(response.status_code)
```

Figura 1.6 – „ParametruTimeoutHelp”

Exercițiul 7. Cronometrarea unei cereri: Măsurarea precisă a duratei de execuție a programului apelând time.perf_counter() înainte și după trimiterea cererii web, comparând valoarea cu proprietatea internă elapsed.total_seconds().

```python
# Exercitiul 7
import time
timp1=time.perf_counter()
print(timp1)
response=requests.get(BASE_URL, timeout=TIMEOUT)
timp2=time.perf_counter()
print(timp2)
durata=timp2-timp1
print(durata)
print(response.elapsed)
```

Figura 1.7 – „MăsurareTimpExecuție”

Exercițiul 8. Când un import eșuează: Protejarea programului împotriva prăbușirii (crash) la lipsa modulului beautifulsoup4 (bs4) prin includerea comenzii de import într-un bloc defensiv de tip try / except ImportError.

```python
# Exercitiul 8
try:
    import bs4
    print("bs4 version:", bs4.__version__)
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```

Figura 1.8 – „TratareEroareImport”

Capitolul 2: Metode și atribute ale obiectelor web

Exercițiul 9. Elementele de bază ale răspunsului: Extragerea informațiilor esențiale din răspuns (codul de stare HTTP, validitatea, URL-ul, formatul text) și diferențierea corectă între metode și atribute prin funcția callable().

```python
# Exercitiul 9

response=rq.get(BASE_URL,timeout=TIMEOUT)
print(response.status_code)  #atribut
print(response.ok)           #atribut
print(response.url)          #atribut
print(response.encoding)     #atribut
```

Figura 2.1 – „AtributeRăspunsHTTP”

Exercițiul 10. Tratarea erorilor cu o metodă: Protejarea rulării prin apelarea r.raise_for_status() pentru rute false (pagini inexistente) și captarea controlată a erorilor din clasa requests.HTTPError.

```python
# Exercitiul 10
response=rq.get(BASE_URL, timeout=TIMEOUT)
response.raise_for_status
try:
    response=rq.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
    response.raise_for_status
except response.HTTPError:
   print("Pagina nu a fost gasita")
```

Figura 2.2 – „TratareExcepțieHTTPError”

Exercițiul 11. Toate antetele: Iterarea structurată prin colecția complexă de date returnată de response.headers.items() pentru obținerea de perechi cheie-valoare specifice conexiunii serverului.

```python
# Exercitiul 11

response=rq.get(BASE_URL,timeout=TIMEOUT)
for nume, valoare in response.headers.items():
    print(nume, ":", valoare)
```

Figura 2.3 – „IterareAnteteHeaders”

Exercițiul 12. Metode de dicționar: Căutarea în antete folosind metoda nativă get() cu răspuns de rezervă (fallback default) pentru "Content-type" și dovedirea faptului că acestea nu sunt sensibile la tipul caracterelor (case-insensitive).

```python
# Exercitiul 12

response=rq.get(BASE_URL,timeout=TIMEOUT)
print(response.headers.get("Server" , "Lipseste"))
print(response.headers.get("Content-Type" , "Lipseste"))
#observ ca indiferent daca folosesc majuscule sau minuscule valorile vor fi
#aceleasi, deci HTTP u nu e case sensitive
print(response.headers.get("content-type"))
```

Figura 2.4 – „MetodeDicționarHeaders”

Exercițiul 13. Înlănțuirea metodelor de șir: Găsirea cuvântului țintă în sursa text a paginii prin combinarea și înlănțuirea consecutivă a metodelor de string (.lower().count()).

```python
# Exercitiul 13
response=rq.get(BASE_URL,timeout=TIMEOUT)
numar=response.text.lower().count("cyber")
#lower() se foloseste pentru a transforma majuscule in minuscule, returnand un string
#iar string ca tip de date are optiunea de count()
print(numar)
```

Figura 2.5 – „ÎnlănțuireMetodeString”

Exercițiul 14. Găsirea titlului: Izolarea și decuparea textului din etichetele HTML de tip <title> folosind tăierea (slicing) pozițională combinată cu find() și curățarea spațiilor cu strip().

```
# Exercitiul 14

response=rq.get(BASE_URL,timeout=TIMEOUT)
start=response.text.find("<title>") + len("<title>")
end=response.text.find("</title>")
titlu=response.text[start:end].strip()
print(titlu)
```

Figura 2.6 – „Gasirea titlului”

Exercițiul 15. Numărarea liniilor: Analiza cantitativă a structurii paginii despărțind codul HTML pe rânduri distincte cu splitlines() și identificarea secvenței de text cu cea mai mare lungime.

```
# Exercitiul 15

response=rq.get(BASE_URL,timeout=TIMEOUT)
linii=response.text.splitlines()
print("Numar de linii:", len(linii))
max=max(linii, key=len)
```

Figura 2.7 – „NumărareLiniiHTML”

Exercițiul 16. Verificarea HTTPS: Interogarea nivelului de securitate al protocolului verificând condițional prezența șirului de caractere „https://” la debutul string-ului prin utilizarea metodei .startswith().

```
# Exercitiul 16

response=rq.get(BASE_URL,timeout=TIMEOUT)
if response.url.startswith("https://"):
    print ("Conexiune securizata")
else:
    print ("Conexiune nesecurizata")
```

Figura 2.8 – „VerificareProtocolHTTPS”

Exercițiul 17. Urmărirea redirecționărilor: Verificarea traseului rețelei, procesând atributul .history pentru a afișa schimbările progresive de coduri și redirecționările URL până la destinația finală.

```python
# Exercitiul 17

response=rq.get("http://cybercor.org ",timeout=TIMEOUT)
for redirect in response.history:
    print(redirect.status_code, redirect.url)

print("URL final:", response.url)
```

Figura 2.9 – „UrmărireRedirecționăriHTTP”

Exercițiul 18. HEAD versus GET: Contrastul volumului descărcat de date calculând lungimea conținutului; cererea GET a returnat tot corpul, iar cererea HEAD a descărcat, în mod predictibil, doar metadatele (rezultat lungime: 0).

```python
# Exercitiul 18

response=rq.head(BASE_URL,timeout=TIMEOUT)
print(len(response.content))
#0 deoarece HEAD nu returneaza continutul paginii, doar header-ul
response=rq.get(BASE_URL,timeout=TIMEOUT)
print(len(response.content)) #returneaza lungimea continutului paginii
```

Figura 2.10 – „ComparațieGETvsHEAD”

Exercițiul 19. Cookie-uri: Parcurgerea obiectului cookies primit în răspunsul cererii web și analizarea proprietăților .name și nivelul de siguranță .secure al acestora.

```python
# Exercitiul 19

response=rq.get(BASE_URL,timeout=TIMEOUT)
if response.cookies:
    for cookie in response.cookies:
        print( "Nume:", cookie.nume)
        print( "Secure:", cookie.secure)
else:
    print("Nu exista cookies")
```

Figura 2.11 – „InspectareObiecteCookies”

Exercițiul 20. Sesiuni: Utilizarea obiectului avansat requests.Session() pentru a simula identitatea și atributele de browser (schimbând "User-Agent-ul" default) păstrând parametrii pentru mai multe conexiuni și analizând formatul .json().

```python
# Exercitiul 20

session = rq.Session()
session.headers.update({"User-Agent": "WebLab-Snecs"})
response = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
print(response.json())  #json() este o metoda care returneaza un dictionar
```

Figura 2.20 – „ConfigurareSesiuneUserAgent”

Capitolul 3: Scrierea propriilor funcții

Exercițiul 21. Prima funcție: Crearea și abstractizarea funcției fetch(url) concepută cu limită obligatorie de timp pentru executarea conexiunilor la adrese dinamice.

```python
# Exercitiul 21

def fetch(url, timeout=TIMEOUT):
    return rq.get(url, timeout=timeout)


response = fetch(BASE_URL)
print(response.status_code)
```

Figura 3.1 – „DefinireFuncțieFetch”

Exercițiul 22. Returnarea unei singure valori: Construirea funcției get_status care procesează un flux, preia și returnează pur numeric un cod de operare din conexiune, fiind apelată în buclă cu time.sleep(1).

```python
# Exercitiul 22

def get_status(url, timeout=TIMEOUT):
    response = rq.get(url, timeout=timeout)
    return response.status_code

print(get_status(BASE_URL))
time.sleep(1)  # Pauză de 1 secundă
print(get_status(BASE_URL + "/robots.txt"))
time.sleep(1)  # Pauză de 1 secundă
print(get_status(BASE_URL + "/sitemap.xml"))
```

Figura 3.2 – „FuncțiaGetStatus”

Exercițiul 23. Parametri impliciți: Actualizarea definiției funcției din primul exercițiu prin setarea unui timeout prestabilit (default parameter) egal cu 10 secunde.

```
# Exercitiul 23

def fetch(url, timeout=TIMEOUT):
    return rq.get(url, timeout=timeout)
response=fetch(BASE_URL)
print(response.status_code)
response=fetch(BASE_URL, timeout=3)
print(response.status_code)
```

Figura 3.3 – „ParametriImplicițiTimeout”

Exercițiul 24. O singură sarcină pe funcție: Construirea funcției get_title(html) separând complet cererea (request-ul) de rețea de operațiunea efectivă de procesare și extragere din text.

```
# Exercitiul 24

def get_title(url, timeout=TIMEOUT):
    response = rq.get(url, timeout=timeout)
    start = response.text.find("<title>") + len("<title>")
    end = response.text.find("</title>")
    return response.text[start:end].strip()

titlu = get_title(BASE_URL)
print(titlu)
```

Figura 3.4 – „FuncțiaGetTitleSeparata”

Exercițiul 25. Docstring-uri: Realizarea autodescrierii funcțiilor din modul adăugând comentarii specializate (""" ... """) care pot fi citite programatic ulterior folosind comanda help().

```
# Exercitiul 25

def get_title(html):
    """Returnează titlul paginii din șirul HTML."""
    start = html.find("<title>") + len("<title>")
    end = html.find("</title>")
    return html[start:end]

print(get_title.__doc__)  # afișează docstring-ul funcției
```

Figura 3.5 – „DocumentareDocstringFuncție”

Exercițiul 26. Adnotări de tip: Declararea de indicii de tipărit (type hints: -> int și url: str) menite să arate utilizatorului natura valorilor așteptate fără blocarea execuției la valori greșite.

```
# Exercitiul 26

def get_status(url: str) -> int:
    """Returnează codul de stare HTTP al răspunsului pentru URL-ul dat."""
    response = rq.get(url, timeout=TIMEOUT)
    return response.status_code
print(get_status("https://cybercor.org"))  # Funcționează corect
# print(get_status(123))
# Mă oprește o eroare de tip TypeError, deoarece funcția așteaptă un argument de tip str, dar a primit un int.
```

Figura 3.6 – „AdnotăriTipTypeHints”

Exercițiul 27. Fără opriri neașteptate: Prevenirea generală a oricăror prăbușiri (crash) cauzate de nume de domeniu false sau rețele indisponibile, forțând prin blocuri try/except un retur boolean fals curat din funcția page_exists.

```
# Exercitiul 27

def page_exists(url):
    try:
        response = rq.get(url, timeout=TIMEOUT)
        return response.status_code == 200
    except rq.RequestException:
        return False
print(page_exists(ECHO_URL))
print(page_exists("https://this-domain-does-not-exist.invalid"))
```

Figura 3.7 – „FuncțiaPageExistsRegulată”

Exercițiul 28. Returnarea unui dicționar: Iterarea pe o listă predefinită de URL-uri (cum ar fi robots.txt, sitemap.xml) și completarea dinamică a unui dicționar structural final mapând rutele locale la codurile de execuție din răspuns.

```
# Exercitiul 28

def check_paths(base, paths):
    """Verifică dacă paginile pentru căile date există pe serverul specificat."""
    result = {}
    for path in paths:
            response = rq.get(base + path , timeout=TIMEOUT)
            result[path] = response.status_code

    return result

print(check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))
```

Figura 3.8 – „FuncțiaCheckPathsDicționar”

Exercițiul 29. Argumente cu nume: Scrierea funcției get_header cu o cerință de valoare de rezervă (default) apelată clarificând manual poziția datelor prin argumente cheie (ex. name="Server").

```
    # exercitiul 29
print("Exerciti (parameter) url: Unknown
def get_header(url, name, default="lipsește"):
    r = requests.get(url, timeout=TIMEOUT)
    return r.headers.get(name, default)
print(get_header(BASE_URL, name="Server"))
```

Figura 3.9 – „ArgumenteCuNumeHeader”

Exercițiul 30. Antete de securitate: Validarea în interiorul dicționarului a prezenței anumitor antete esențiale folosite de browser pentru conexiuni criptate și apărarea contra atacurilor cibernetice (ex. Referrer-Policy, Strict-Transport-Security).

```
# Exercitiul 30

def security_headers(url):
    """Returnează un dicționar cu header-ele de securitate pentru URL-ul dat."""
    response = rq.get(url, timeout=TIMEOUT)
    headers = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Content-Type-Options",
        "X-Frame-Options",
        "Referrer-Policy",
    ]
    return {header: header in response.headers for header in headers}

print(security_headers(BASE_URL))
```

Figura 3.10 – „FuncțiaSecurityHeaders”

Exercițiul 31. Funcții care apelează funcții: Procesarea matematică a elementelor din interiorul dicționarului returnat la pasul 30, prelucrând datele cu funcțiile utilitare sum() și len() pentru a crea sintetic un procent/raport.

```
# Exercitiul 31
def score_headers(headers):
    """Calculează scorul de securitate pe baza header-elor de securitate."""
    total = len(headers)
    gasite=sum(headers.values())
    return print("Scorul:", gasite, "/", total, "headere de securitate găsite.")

score_headers(security_headers(BASE_URL))
```

Figura 3.11 – „CalculScorAntete”

Exercițiul 32. Două funcții, o sarcină: Scrierea a două seturi logice distincte – prima funcție, fetch_robots, accesează o resursă brută din rețea; a doua secțiune, disallowed_paths, analizează textul împărțind șirul (cu funcția split) conform setărilor directivei "Disallow:".

```
# Exercitiul 32

def fetch_robots_txt(url):
    """Returnează conținutul fișierului robots.txt pentru URL-ul dat."""
    response = rq.get(url + "/robots.txt", timeout=TIMEOUT)
    if response.status_code == 200:
        return response.text
    else:
        return None

def disallowed_paths(robots_txt):
    """Returnează o listă cu căile interzise din fișierul robots.txt."""
    paths = []
    for line in robots_txt.splitlines():
        if line.startswith("Disallow:"):
            path = line.split(":", 1)[1].strip()
            paths.append(path)
    return paths
print(fetch_robots_txt(BASE_URL))
print(disallowed_paths(fetch_robots_txt(BASE_URL)))
```

Figura 3.12 – „ParsareRobotsDisallowed”

Exercițiul 33. Oricâte argumente: Utilizarea capabilității *urls (args variable arguments parameter) pentru a accepta fără limită de elemente orice număr de surse transmise ca parametri într-o singură execuție de verificare a vitezei de ping.

```
# Exercitiul 33

def response_times(*urls):
    """Returnează un dicționar cu timpii de răspuns pentru fiecare URL dat."""
    times = {}
    for url in urls:
        start=time.time()
        rq.get(url, timeout=TIMEOUT)
        end=time.time()

        times[url] = end - start
    return times
print(response_times(BASE_URL, ECHO_URL, "https://www.google.com"))
```

Figura 3.13 – „ArgumenteVariabileArgs”

Exercițiul 34. Argumente cu nume colectate: Trimiterea și procesarea dictărilor dinamice utilizând structura universală **details (**kwargs), despachetând informațiile cheie și valoare și combinându-le direct într-un string (text) formator pentru loguri (evidența erorilor).

```
# Exercitiul 34

def log(message, **detalii):
    """Afișează un mesaj de log cu detalii suplimentare."""
    print(message, end="")
    for mes,det in detalii.items():
        print(f" | {mes}= {det}" , end="")
log("Verificat", url=BASE_URL, status=get_status(BASE_URL))
```

Figura 3.14 – „ArgumenteKeywordKwargs”

Capitolul 4: Alte module din biblioteca standard

Exercițiul 35. Descompunerea unui URL: Secționarea corectă și completă a unui șir web (de exemplu schemă, cale, parametru interogare) utilizând obiectul urlparse provenit din biblioteca preinstalată urllib.parse.

```
# Exercitiul 35
import urllib.parse
def descompunere(url):
    """Descompune un URL în componentele sale."""
    rezultat = urllib.parse.urlparse(url)
    return {
        "scheme": rezultat.scheme,
        "netloc": rezultat.netloc,
        "path": rezultat.path,
        "query": rezultat.query,
        "fragment": rezultat.fragment
    }
# print(descompunere("https://cybercor.org/path?x=1#top"))
```

Figura 4.1 – „DescompunereURLParse”

Exercițiul 36. Compunerea URL-urilor: Recrearea adreselor online accesibile pornind de la căi locale (relative) și domeniul părinte de bază folosind instrumentul eficient de combinare urljoin.

```python
# Exercitiul 36

def compunerea(url, part):
    """Recompune un URL din componentele sale."""
    rezultat = urllib.parse.urljoin(url,part)
    return rezultat
# print(compunerea(BASE_URL, "/about"))
# print(compunerea(BASE_URL, "contact.html"))
# print(compunerea(BASE_URL, "../index.html"))
```

Figura 4.2 – „CompunereURLJoin”

Exercițiul 37. Extragerea legăturilor: Căutarea adâncă și extragerea adreselor web prin folosirea motorului Python de expresii regulate re.findall, aplicând modelul de etichetă HTML href.

```python
# Exercitiul 37

def extract_links(html):
    """Extrage toate link-urile dintr-o pagină web."""
    links=re.findall(r'href="([^"]+)"', html)
    return list(set(links))
html=rq.get(BASE_URL).text
# print(extract_links(html))
```

Figura 4.3 – „ExtragereLinkuriRegex”

Exercițiul 38. Interne sau externe?: O rutină de comparație în care colecția principală de rute (URL-uri) extrasă la punctul anterior este validată pentru a grupa link-urile destinate site-ului propriu (same_domain) versus adresele către alte platforme (other_domains).

```python
# Exercitiul 38

def split_links(links, domain):
    """Împarte link-urile în absolute și relative."""
    interne = []
    externe = []
    for link in links:
        url=urllib.parse.urljoin(domain, link)
        if urllib.parse.urlparse(url).netloc == urllib.parse.urlparse(domain).netloc:
            interne.append(url)
        else:
            externe.append(url)
    return interne, externe
# print(split_links(extract_links(html), BASE_URL))
```

Figura 4.4 – „SeparareLinkuriDomeniu”

Exercițiul 39. Propriile metode: Moștenirea, din cadrul bibliotecii html.parser, a clasei fundamentale HTMLParser prin crearea clasei proprii (child class) ImageFinder. Aceasta suprascrie metoda nativă handle_starttag pentru identificarea atributelor de imagini (src) din structura paginii web.

```
# Exercitiul 39
from html.parser import HTMLParser
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""
class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)
finder = ImageFinder()
finder.feed(test_html)
print(finder.images)
assert finder.images == [
    "/logo.png",                          # imagine obișnuită
    "poza.jpg",                           # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Figura 4.5 – „ClasaImageFinderHTMLParser”

Exercițiul 40. Amprenta paginii: Protejarea identității sau verificarea modificării datelor în timp prin generarea codificării tip hashing pe un bloc de bytes brut aparținând structurii statice sau dinamice a serverului utilizând funcția criptografică hashlib.sha256().hexdigest().

```
# Exercitiul 40

def page_fingerprint(url):
    """Returnează un fingerprint al paginii web pentru URL-ul dat."""
    return hashlib.sha256(response.content).hexdigest()
# print(page_fingerprint(BASE_URL))
# print(page_fingerprint(BASE_URL))
# O sa fie diferite in cazul in care se va schimba continutul paginii între cele două apeluri
```

Figura 4.6 – „AmprentăDigitalăSHA256”

Exercițiul 41. Salvarea în JSON: Formatarea compatibilă și serializarea datelor native de tip dicționar (precum antetele rețelei obținute) stocând rezultatul prin metoda json.dump **într-un format de fișier fizic lizibil; procedură secondată de procesul de recuperare cu** json.load()**.**

```
# Exercitiul 41

def salvare_headers(response, filename):
    """Salvează headerele HTTP într-un fișier JSON."""
    with open(filename, "w") as file:
        json.dump(dict(response.headers), file, indent=2)
    with open(filename, "r") as file:
        headers = json.load(file)
    return headers
# print(salvare_headers(response, "headers.json"))
```

Figura 4.7 – „SalvareIncarcareJSON”

Exercițiul 42. Interogare DNS: Descoperirea directă a locației fizice a echipamentelor ce rulează serverele rețelelor folosind serviciul Domain Name System (DNS) din pachetul IP socket.gethostbyname.

```
# Exercitiul 42

def resolve(hostname):
    """Rezolvă un hostname în adresa sa IP."""
    import socket
    return socket.gethostbyname(hostname)
# print(resolve("cybercor.org"))
# Ar trebui să returneze adresa IP a domeniului
```

Figura 4.8 – „InterogareDNSGetHostByName”

Exercițiul 43. Expirarea certificatului: Conexiunea la portul de siguranță portul 443 al mașinii gazdă, capturarea prin modulul ssl **a validității (**notAfter**) și conversia din text standard (format timp server) în zile terestre libere până la degradare.**

```
# Exercitiul 43

def cert_days_left(hostname):
    """Returnează numărul de zile rămase până la expirarea certificatului SSL al unui hostname."""
    import ssl
    import socket
    import datetime

    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()

    zile_ramase=cert['notAfter']
    return (ssl.cert_time_to_seconds(zile_ramase) - time.time()) / (24 * 60 * 60)
# Convertim secunde în zile
# print(cert_days_left("cybercor.org"))
# # Ar trebui să returneze numărul de zile rămase până la expirarea certificatului SSL
```

Figura 4.9 – „VerificareExpirareCertificatSSL”

Capitolul 5: Construirea propriului modul și proiectul final

Exercițiul 44. Crearea unui modul: Arhitecturarea și mutarea componentelor esențiale într-un fișier logic dedicat denumit webtools.py, lăsând scriptul curent main.py foarte simplificat prin simplul import al bibliotecii create manual de către noi (import webtools).

```
# Exercitiul 44

import webtools
print(webtools.get_title(BASE_URL))
```

Figura 5.1 – „ModulWebToolsSiMain”

Exercițiul 45. Importarea anumitor nume: Folosirea notației avansate de chemare de obiect from webtools import... și clarificarea, print-un comentariu explicit, a felului în care spațiul de nume (namespace-ul) soluționează problemele unde funcțiile importate ar avea o coliziune (conflict direct) de vizibilitate sau funcție pe nume echivalent din cod.

```
# Exercitiul 45
from webtools import get_title
from webtools import security_headers
# Daca main.py ar avea o functie get_title, aceasta ar fi apelata in locul functiei din webtools.py.
```

Figura 5.2 – „ImportSelectivFromWebTools”

Exercițiul 46. Blocul main: Structurarea protecției testelor individuale (unit testing) doar către programul modul de origine (locația mamă), executând instrucțiuni exclusiv când parametrul intern pre-instalat nativ de Python are formă explicită de script apelat (la fel cum definește variabila globală internă specială if __name__ == "__main__":).

```
# Exercitiul 46

# Autotestul lucreaza doar in primul caz deoarece webtools.py este folosit direct
# pe cand in al doilea caz webtools este importat.
```

Figura 5.3 – „BloculMainNameTest”

Exercițiul 47. Constante de modul: Setarea dicționarului constant DEFAULT_HEADERS cu scop general valabil (globally scoped variable) pe primele rânduri din fișierul importat de tip utilitar webtools.py pentru referențierea valorilor statice din toate sub-funcțiile create de utilizator.

```
# Exercitiul 47

from webtools import DEFAULT_HEADERS
print(DEFAULT_HEADERS)
```

Figura 5.4 – „ConstantaDefaultHeaders”

Exercițiul 48. Argumente din linia de comandă: Interacțiunea directă din terminal transformând scriptul-ul într-un tool complet operațional de linia de comandă (CLI). Prin importul pachetului intern standardizat argparse, codul primește adresa link din sintaxa curentă (python main.py URL) transmițând datele direct intern, înlocuind linkurile blocate la modul static anterior (variabila constantă BASE_URL fiind astfel ștearsă).

```
# Exercitiul 48
import argparse
from webtools import get_status
parser=argparse.ArgumentParser()
parser.add_argument("url")
args=parser.parse_args()
print(get_status(args.url))
```

Figura 5.5 – „PreluareArgumenteArgparse”

Exercițiul 49. Un raport CSV: Scrierea datelor ciclice stocate din variabile (adunate la extragerea adreselor web) într-o formatare secvențială tabelară sub formă de text pur (extensia .csv), inserând ca atribut de control ora generării în forma universală ISO 8601 (datetime.now().isoformat()).

```python
# Exercitiul 49
import csv
from datetime import datetime
from part3_functions import check_paths as cp
rezultat=cp()
with open("report.csv" , "w") as file:
    WRITER=csv.writer(file)
    WRITER.writerow(["Path", "Status" , "Checked_At"])
    for path, status in rezultat.items():
        WRITER.writerow([path, status, datetime.now().isoformat()])
print("Raportul a fost salvat în report.csv")

#print(loader)
```

Figura 5.6 – „GenerareRaportCSV”

Exercițiul 50. Proiect final: raportul site-ului: Consolidarea tuturor elementelor implementate și testate până în acest pas (de la extragere date la verificare HTTPS/SSL și generare structură) creând funcția complexă site_report(url) situată integrat la sursa modulului webtools.py. Datele se consolidează logic și estetic, cu afișare către operatorul direct și exportare permanentizată în dicționare .json localizate exact conform instrucțiunilor.

```python
# Exercitiul 50
def site_report(url):
    """Generează un raport al site-ului și îl salvează într-un fișier CSV."""
    from part3_functions import check_paths as cp
    from datetime import datetime
    import csv
    # Cod de stare si URL final
    r=webtools.fetch(url)
    status_code=r.status_code
    # Titlu
    title=webtools.get_title(url)
    # Adresa IP
    ip_address=webtools.resolve(url)
    # Redirecționări
    redirects=[]
    for redirect in r.history:
        redirects.append((redirect.status_code, redirect.url))
    # Scorul de securitate
    headers=webtools.security_headers(url)
    security_score=sum(headers.values())
    # Zile rămase până la expirarea certificatului SSL
    days_left=webtools.cert_days_left(url)
    # Legaturi externe si interne
    external_links, internal_links=webtools.extract_links(url)
    # Caile interzise din robots.txt
    robots_txt=webtools.fetch_robots_txt(url)
    disallowed_paths=webtools.disallowed_paths(robots_txt)
    # Afisarea raportului
    print(f"Raport pentru {url}:")
    print(f"Cod de stare: {status_code}")
    print(f"Titlu: {title}")
    print(f"Adresa IP: {ip_address}")
    print(f"Redirecționări: {redirects}")
    if redirects:
        for redirect in redirects:
            print(f" {redirect[0]} -> {redirect[1]}")
    print(f"Scor de securitate: {security_score} din {len(headers)} headere de securitate găsite.")
    print(f"Zile rămase până la expirarea certificatului SSL: {days_left:.2f}")
    print(f"Legături externe: {external_links}")
    print(f"Legături interne: {internal_links}")
    print(f"Căi interzise din robots.txt: {disallowed_paths}")
    raport_json = {
            "url": url,
            "status_code": status_code,
            "title": title,
            "ip_address": ip_address,
            "redirects": redirects,
            "security_score": security_score,
            "days_left_ssl": days_left,
            "external_links": external_links,
            "internal_links": internal_links,
            "disallowed_paths": disallowed_paths
        }
    with open("report.json", "w") as file:
        json.dump(raport_json, file)
    print("Raportul a fost salvat in report.json")
```

Figura 5.7 – „FuncțiaSiteReportJSON”

Concluzie

În urma realizării acestei lucrări de laborator, a fost demonstrată utilitatea și eficiența limbajului Python în construirea de instrumente dedicate interacțiunii și analizei web. Prin rezolvarea cu succes a celor 50 de sarcini practice, au fost asimilate și aplicate principii fundamentale pentru dezvoltarea de software robust: tratarea excepțiilor de rețea, respectarea eticii de interogare a serverelor externe (prin implementarea limitelor de timp și a pauzelor de execuție) și parsarea eficientă a structurilor de date (HTML, dicționare, liste, JSON).